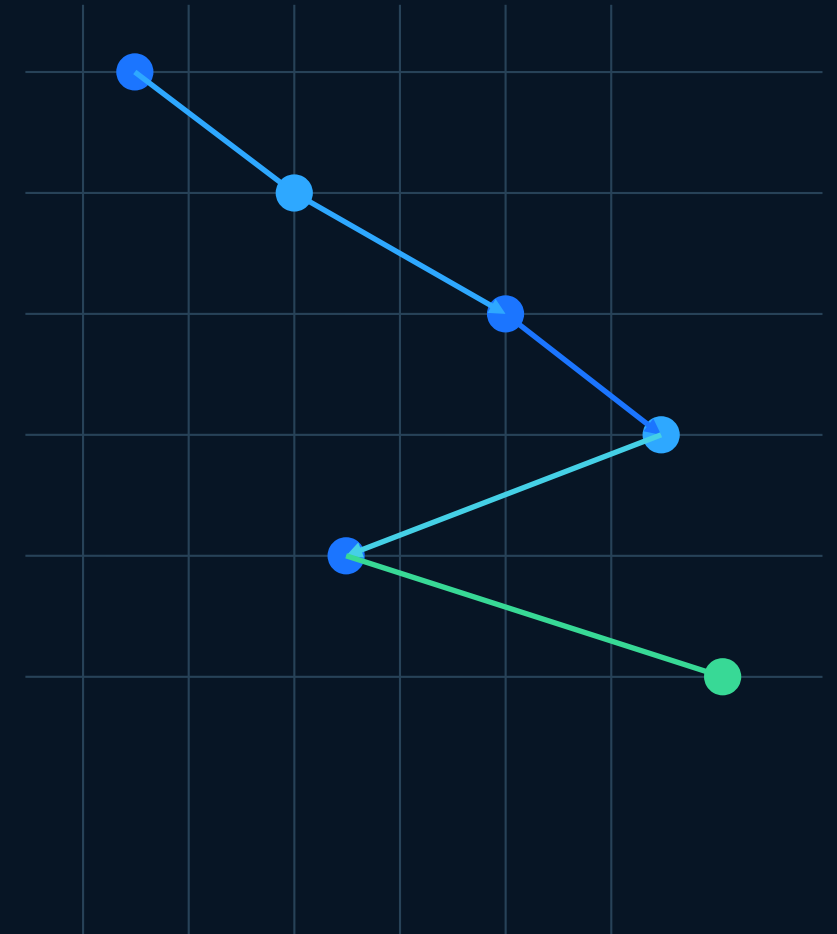


AI Java Engineer

Fábrica gobernada de agentes para desarrollo Java

De un requerimiento en lenguaje natural a artefactos Java/Spring Boot validados, trazables y preparados para revisión humana.

Versión declarada	Host	Target
0.1.0	Python 3.11+	Java 21 · Spring Boot · Maven



Revisión del repositorio público · 23 sep 2026

github.com/7zradar/ai-java-engineer

Reducir handoffs sin perder control

La automatización propuesta cubre el tramo repetitivo entre entender una feature y dejarla lista para revisión.

HOY

Una feature Java atraviesa múltiples handoffs

- 01 **Interpretar** Convertir necesidad de negocio en criterios técnicos.
- 02 **Diseñar** Alinear la solución con el repositorio existente.
- 03 **Implementar + probar** Código, tests, compilación y correcciones iterativas.
- 04 **Revisar + entregar** Diff, seguridad, aprobaciones y preparación del PR.



PROPUESTA

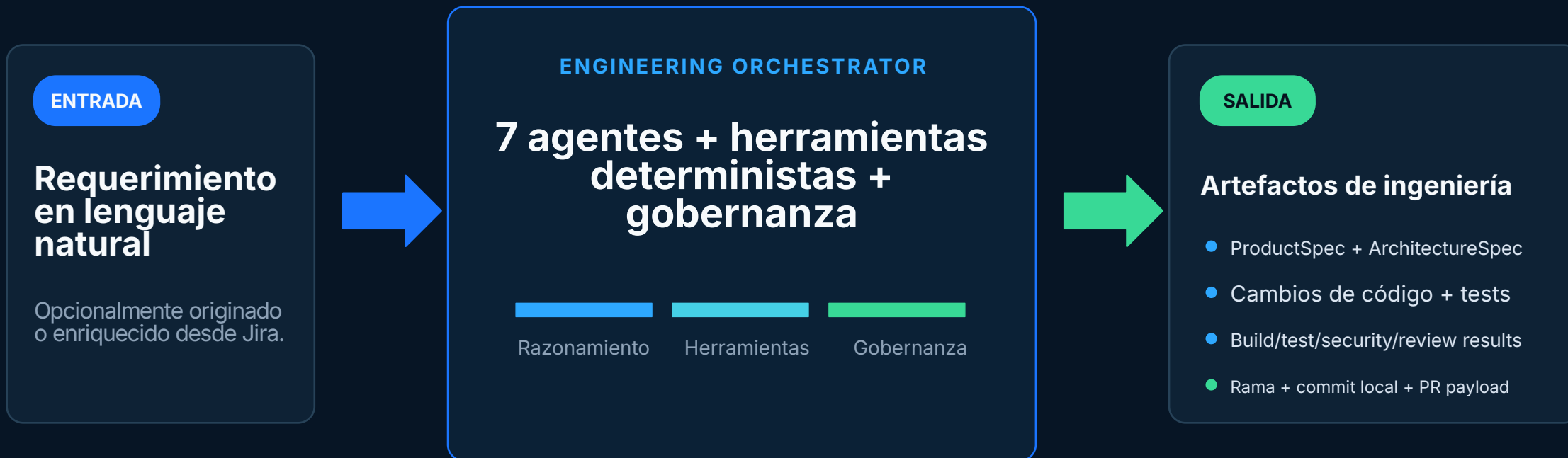
Agentes especializados dentro de un flujo determinista

- 1 **Especialización** Cada agente produce un artefacto o decisión acotada.
- 2 **Determinismo** El orquestador decide el siguiente paso, no un agente libre.
- 3 **Guardrails** Filesystem, seguridad, presupuestos y límites de reintento.
- 4 **Human-in-the-loop** La entrega sensible puede requerir aprobación explícita.

IDEA CLAVE No reemplaza el SDLC: automatiza tramos repetitivos con control explícito y artefactos auditables.

Del requisito a un paquete de cambio revisable

La plataforma ejecuta un estado compartido a través de agentes, herramientas y controles.



SEÑALES DE ALCANCE VERIFICABLE

0.1.0 versión declarada

20

SPECs en estructura

7

agentes instanciados

4

providers implementados

2

backends de ejecución

Tres capas separan creatividad, ejecución y control

El diseño evita que el LLM tenga control directo e irrestricto sobre el pipeline.

ENTRADA

**Requisito
+ contexto
del repo**

no confiable



01

Razonamiento probabilístico

Product · Architect · Coding · QA · Debug · Security · Review

02

Herramientas deterministas

Secure FS · repo scan · Git · build/test adapters · Surefire · SQLite

03

Gobernanza

Permisos · HITL · token/model budgets · reintentos · seguridad · trazabilidad



**ARTEFACTOS
TRAZABLES**

**specs
código
tests
diff
resultados**

MATIZ DE IMPLEMENTACIÓN

README/state mencionan LangGraph, pero graph.py implementa un orquestador secuencial propio y LangGraph no figura como dependencia directa.

Siete especialistas, un único estado de ingeniería

Cada agente tiene una responsabilidad acotada; el orquestador conserva el control del orden.



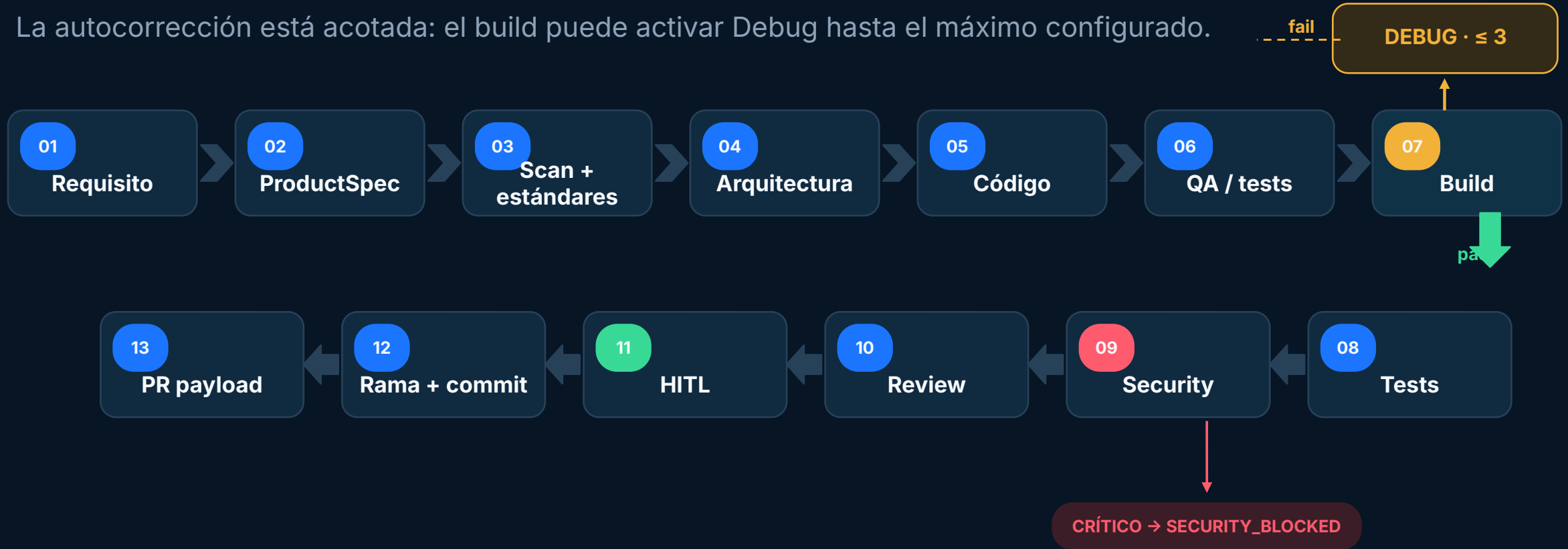
DIFERENCIA CLAVE

QA no "aprueba" el código: genera pruebas de forma independiente. Security puede detener el pipeline. Review consolida evidencia antes de la compuerta humana.

El objetivo es reducir correlación de errores entre generación, prueba, seguridad y revisión.

El flujo de ingeniería es explícito y recuperable

La autocorrección está acotada: el build puede activar Debug hasta el máximo configurado.



**ENTREGA REAL
OBSERVADA**

El flujo crea rama y commit local y construye pr_payload; el código revisado no demuestra la publicación automática del PR en GitHub.

Autonomía acotada por boundaries y compuertas

El repositorio trata requisitos y contenido externo como entrada no confiable.

- 01 Contener** Rutas canónicas y workspace autorizado; path traversal produce violación.
- 02 Sanitizar** SecretSanitizer enmascara API keys, tokens y credenciales.
- 03 Neutralizar** Contenido del repo se encapsula como DATA frente a prompt injection.
- 04 Bloquear** Hallazgo crítico de Security detiene el pipeline.
- 05 Aprobar + auditar** Operaciones sensibles usan HITL; budgets, checkpoints y trazas limitan ejecución.

POLÍTICA DE RIESGO

READ

LOW

PATCH

MEDIUM

PUSH / PR

HIGH

PROD

FORBIDDEN

La política documentada prohíbe el despliegue a producción desde esta plataforma.

Una API Python coordina UX, modelos, ejecución y estado

La experiencia combina dashboard web, conectores empresariales y backends intercambiables.



Nota de implementación:

AnthropicProvider existe, pero la factory API revisada selecciona Gemini/OpenAI/Mock. El tracing es "OpenTelemetry-compatible"; no se observó dependencia directa de OpenTelemetry en pyproject.

Base coherente para piloto; evidencia insuficiente para producción

La lectura debe separar código ejecutable, modo demo/simulado y objetivos aún por validar operacionalmente.

IMPLEMENTADO

- 7 agentes + orquestador secuencial
- Secure FS, sanitización y security gate
- FastAPI + dashboard + cliente Jira
- Checkpoints SQLite y trazabilidad
- Tests/evaluación presentes en la estructura

DEMO / SIMULACIÓN

- MockProvider y LocalMock son defaults
- Jira dispone de dataset/fallback local
- Estado activo también vive en memoria
- Usuarios y contraseñas demo en código
- PR: se construye payload, no publicación

PENDIENTE PARA PRODUCCIÓN

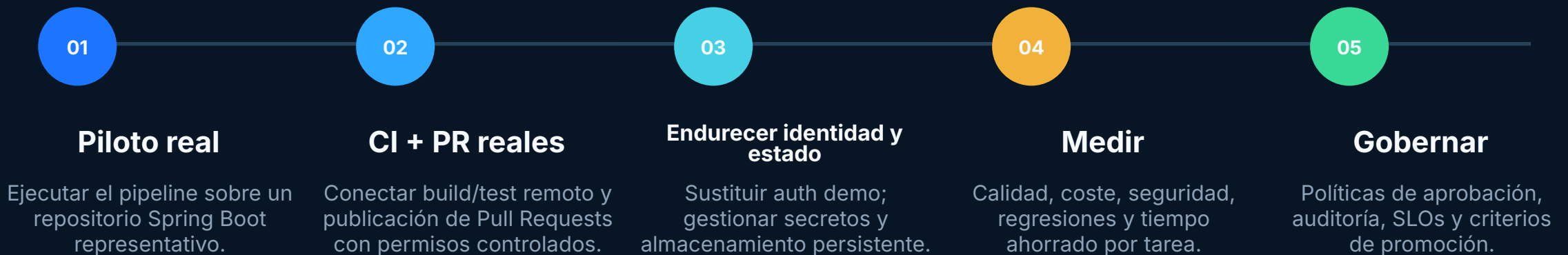
- Validar CI real con repos Spring Boot
- Publicación real de PR y permisos GitHub
- IAM/autenticación y secret management
- Persistencia/HA/operación endurecida
- Benchmarks reales de calidad, coste y seguridad

VERIFICACIÓN

No se afirma que "todas las pruebas pasan": este entorno no pudo clonar el repositorio para ejecutar pytest con PYTHONPATH=src. El estado de la suite queda no verificado.

Convertir una buena demo en evidencia operativa

La prioridad no es añadir más agentes: es cerrar el loop real de entrega, identidad, persistencia y medición.



DECISION GATE

Promover más allá del piloto solo cuando CI, seguridad, identidad, persistencia y métricas estén respaldados por evidencia operacional.

Lectura ejecutiva: la ventaja no es autonomía irrestricta; es autonomía dentro de guardrails verificables.